
Session 12 (AIML) - Assignment Questions

Name: **Vaishnavi Praveen Rathi**

College: **Zeal Polytechnic, Narhe, Pune**

Branch: **Artificial Intelligence and Machine Learning (AIML)**

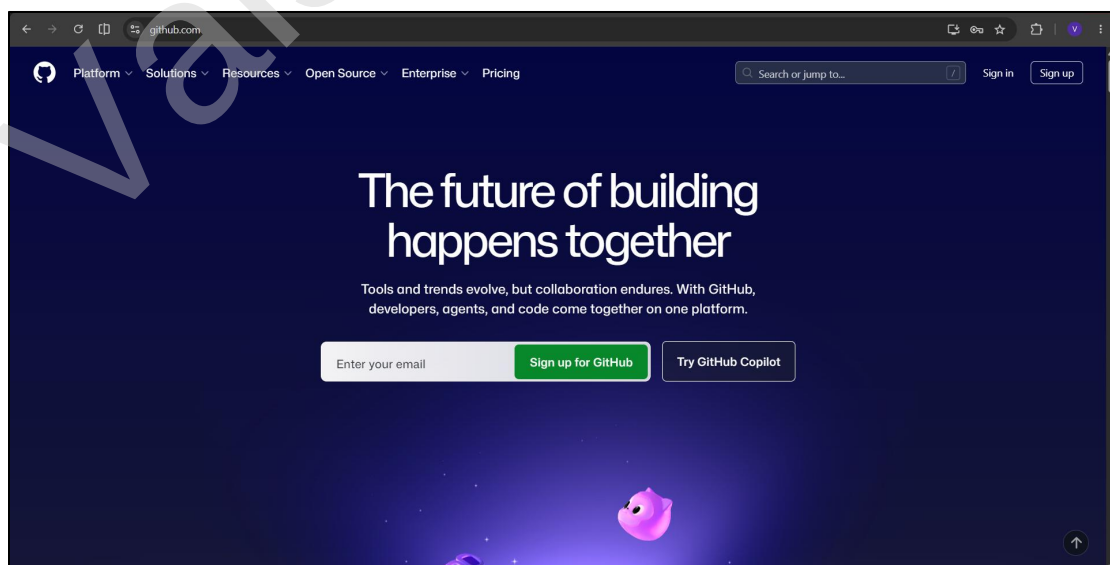
Domain: **AIML (Artificial Intelligence and Machine Learning)**

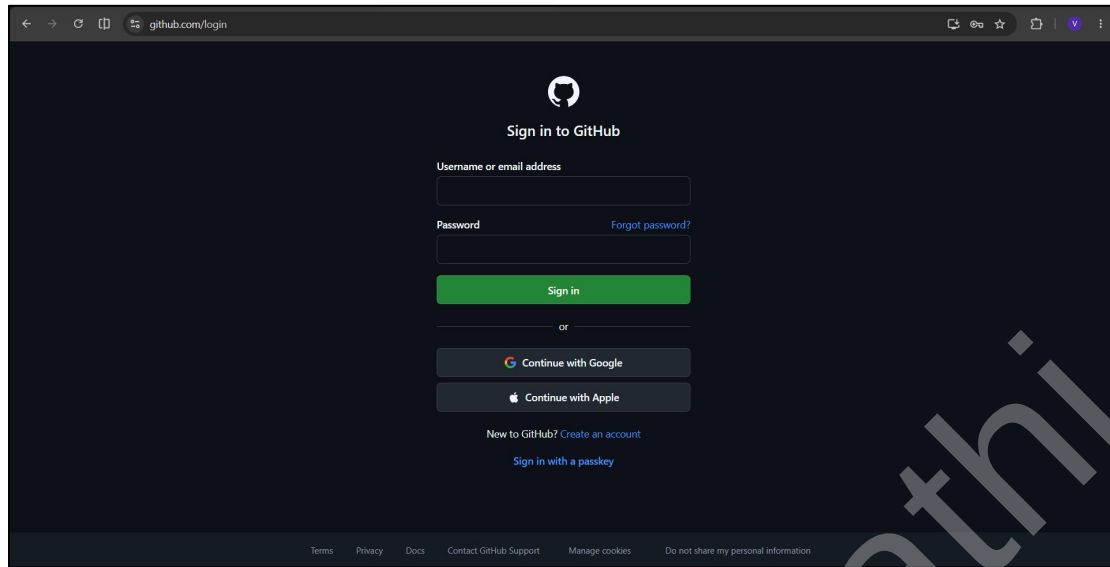
Q1. Github Account Setup

Output Screenshots:

Step 1: Create a GitHub Account

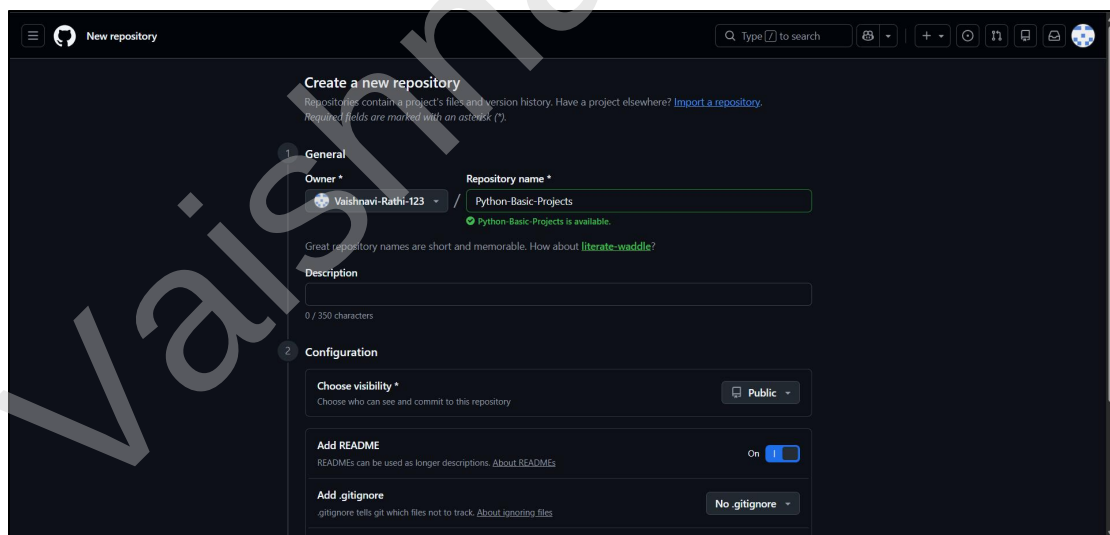
- Go to GitHub.
- Click **Sign Up**.
- Create your account and verify your email.

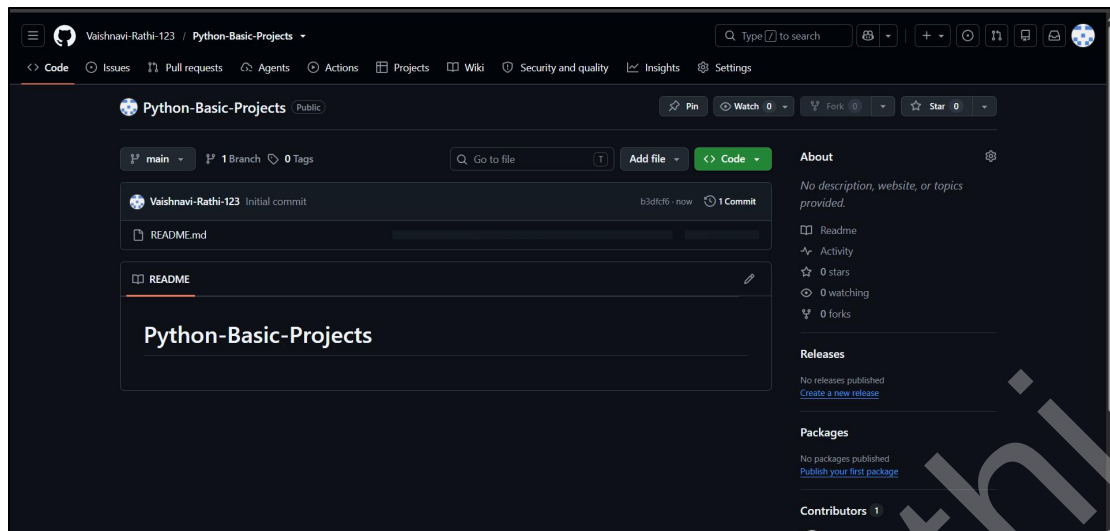




Step 2: Create a New Repository

1. Click the + icon (top-right).
2. Select **New repository**.
3. Repository Name:



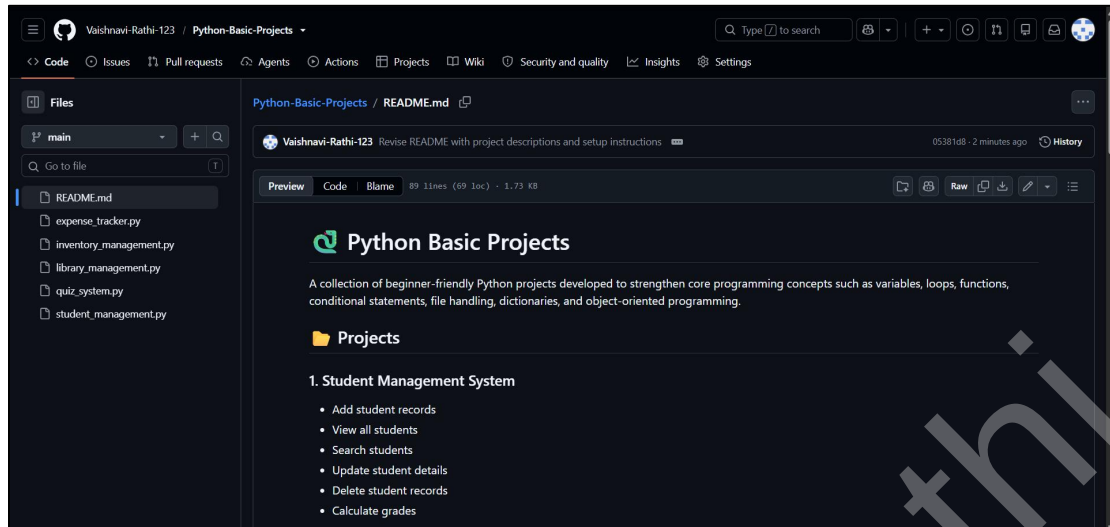


Step 3: Initialize Git in Your Project Folder

Step 4: Connect Your Local Project to GitHub

Step 5: Commit After Each Feature

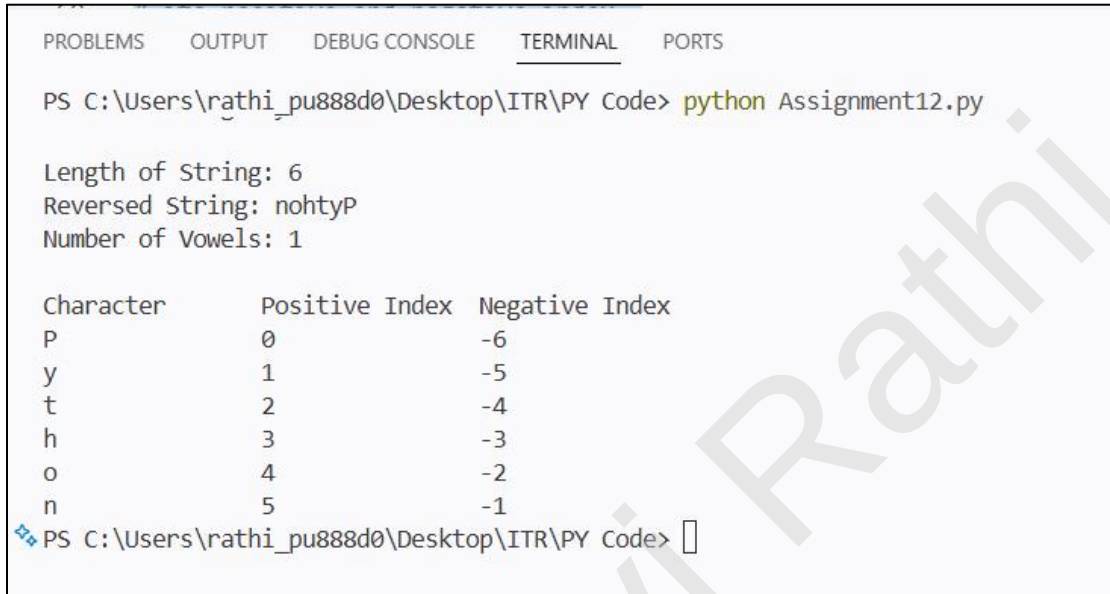
Step 6: Push to GitHub



Github Repo : <https://github.com/Vaishnavi-Rathi-123/Python-Basic-Projects>

Q2. Strings + Loops + Functions

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py

Length of String: 6
Reversed String: nohtyP
Number of Vowels: 1

Character      Positive Index  Negative Index
P              0              -6
y              1              -5
t              2              -4
h              3              -3
o              4              -2
n              5              -1
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """
```

```
'''
```

Q2. (Strings + Loops + Functions)

Write a function `analyze_string(s)` that:

1. Prints the length using `len()`.
2. Prints the string in reverse using slicing.
3. Counts and prints the number of vowels (case-insensitive).
4. Uses a for loop with `range()` to print each character with its positive and negative index.
5. Call the function with user input and handle empty string case.

```
'''
```

```
# Function to analyze the given string
```

```
def analyze_string(s):
```

```
    # Print the length of the string
```

```
    print("\nLength of String:", len(s))
```

```

# Print the string in reverse
print("Reversed String:", s[::-1])

# Count vowels
vowels = "aeiou"
count = 0

for ch in s.lower():
    if ch in vowels:
        count += 1

print("Number of Vowels:", count)

# Print each character with positive and negative index
print("\nCharacter\tPositive Index\tNegative Index")

for i in range(len(s)):
    print(f"{s[i]}\t\t{i}\t\t{i-len(s)}")

# Take input from the user
text = input("Enter a String: ")

# Handle empty string case
if text == "":
    print("Error: Empty string is not allowed.")
else:
    analyze_string(text)

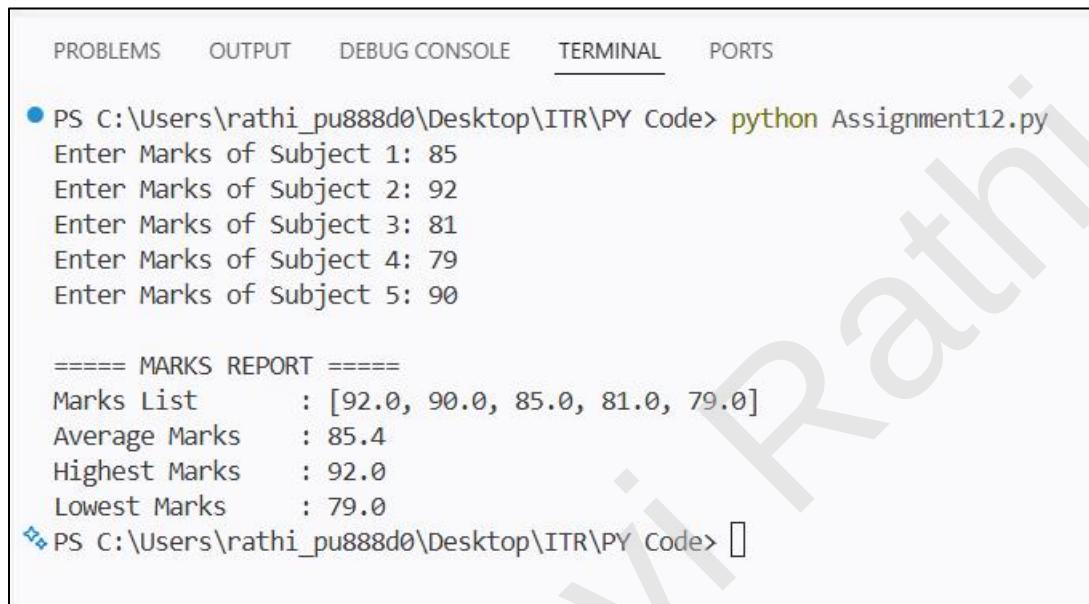
"""Explanation in Comments"""

# The function analyze_string() accepts a string as input.
# len() is used to find the length of the string.
# Slicing [::-1] reverses the string.
# The string is converted to lowercase for case-insensitive vowel counting.
# A for loop with range() prints each character along with
# its positive and negative index.
# The program checks for an empty string before calling the function.

```

Q3. Lists + Functions + Exception Handling

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py
Enter Marks of Subject 1: 85
Enter Marks of Subject 2: 92
Enter Marks of Subject 3: 81
Enter Marks of Subject 4: 79
Enter Marks of Subject 5: 90

===== MARKS REPORT =====
Marks List      : [92.0, 90.0, 85.0, 81.0, 79.0]
Average Marks   : 85.4
Highest Marks   : 92.0
Lowest Marks    : 79.0
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """

'''
Q3. (Lists + Functions + Exception Handling)

Create a function manage_marks() that:

1. Takes 5 subject marks as input from the user.
2. Stores them in a list.
3. Handles ValueError if non-numeric input is given.
4. Calculates and prints the average, highest, and lowest marks.
5. Sorts the list in descending order and prints it.
'''

# Function to manage student marks
def manage_marks():

    # Empty list to store marks
    marks = []

    try:
```



```

# Take marks of 5 subjects
for i in range(1, 6):
    mark = float(input(f"Enter Marks of Subject {i}: "))
    marks.append(mark)

# Calculate average
average = sum(marks) / len(marks)

# Find highest and lowest marks
highest = max(marks)
lowest = min(marks)

# Sort marks in descending order
marks.sort(reverse=True)

# Display the results
print("\n===== MARKS REPORT =====")
print("Marks List      :", marks)
print("Average Marks   :", average)
print("Highest Marks    :", highest)
print("Lowest Marks     :", lowest)

except ValueError:

    # Handle invalid input
    print("Error: Please enter numeric values only.")

# Call the function
manage_marks()

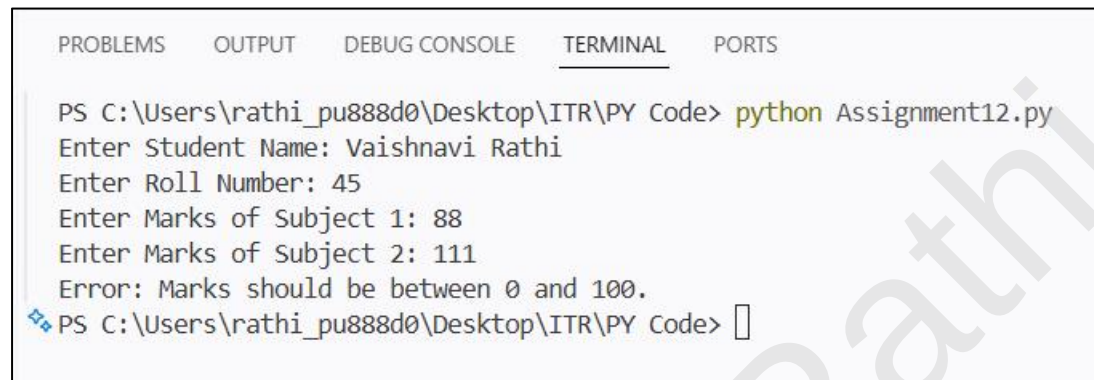
"""Explanation in Comments"""

# The function manage_marks() accepts marks for 5 subjects.
# All marks are stored in a list.
# ValueError is handled if the user enters invalid input.
# sum() and len() are used to calculate the average.
# max() finds the highest mark.
# min() finds the lowest mark.
# sort(reverse=True) arranges the marks in descending order.
# Finally, all results are displayed.

```

Q4. OOP + Lists + Exception Handling

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py
Enter Student Name: Vaishnavi Rath
Enter Roll Number: 45
Enter Marks of Subject 1: 88
Enter Marks of Subject 2: 111
Error: Marks should be between 0 and 100.
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """
```

```
'''
```

Q4. (OOP + Lists + Exception Handling)

Create a class Student with attributes:
name, roll_no, and marks_list.

Methods:

1. `__init__()`
2. `add_mark(mark)`
3. `get_average()`
4. `display_info()`

Create one student object, add marks using user input,
and demonstrate all methods with exception handling.

```
'''
```

```
# Student class
```

```
class Student:
```

```
    # Constructor to initialize student details
```

```
    def __init__(self, name, roll_no):
```

```
        self.name = name
```

```
        self.roll_no = roll_no
```

```
        self.marks_list = []
```

```
    # Method to add marks
```

```

def add_mark(self, mark):

    # Check if marks are valid
    if mark < 0 or mark > 100:
        raise ValueError("Marks should be between 0 and 100.")

    self.marks_list.append(mark)

# Method to calculate average marks
def get_average(self):

    if len(self.marks_list) == 0:
        return 0

    return sum(self.marks_list) / len(self.marks_list)

# Method to display student details
def display_info(self):

    print("\n===== STUDENT DETAILS =====")
    print("Name      :", self.name)
    print("Roll No   :", self.roll_no)
    print("Marks     :", self.marks_list)
    print("Average   :", self.get_average())

try:

    # Take student details
    name = input("Enter Student Name: ")
    roll_no = int(input("Enter Roll Number: "))

    # Create student object
    student = Student(name, roll_no)

    # Take marks for 5 subjects
    for i in range(1, 6):
        mark = float(input(f"Enter Marks of Subject {i}: "))

        # Add marks using class method
        student.add_mark(mark)

    # Display student information
    student.display_info()

except ValueError as e:

    # Handle invalid input or invalid marks
    print("Error:", e)

```

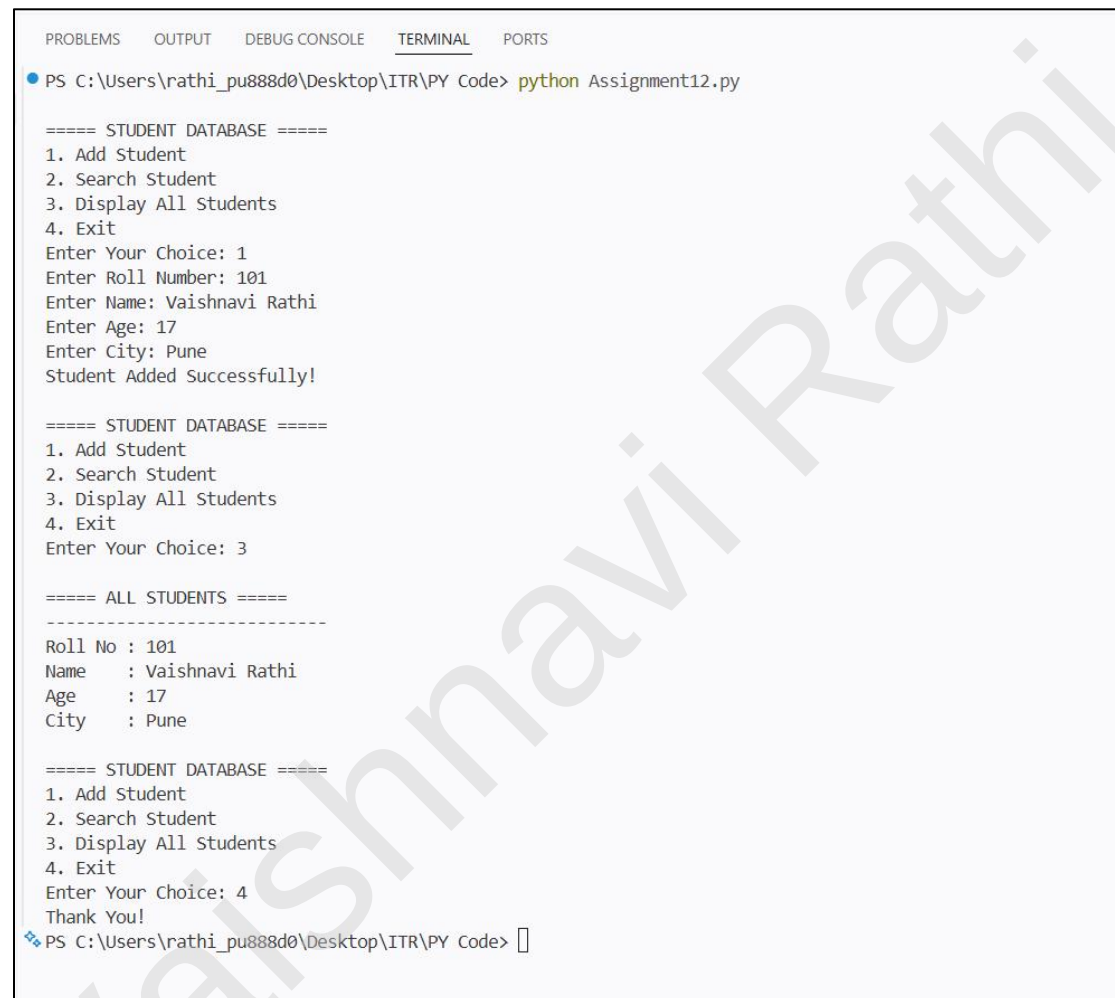
"Explanation in Comments"

```
# The Student class stores student details and marks.  
# __init__() initializes the student's name, roll number, and marks list.  
# add_mark() validates and stores marks in the list.  
# get_average() calculates and returns the average marks.  
# display_info() displays all student details.  
# Exception handling prevents invalid input and invalid marks.
```

Vaishnavi Rathni

Q5. Dictionaries + Functions + Control Structures

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py

===== STUDENT DATABASE =====
1. Add Student
2. Search Student
3. Display All Students
4. Exit
Enter Your Choice: 1
Enter Roll Number: 101
Enter Name: Vaishnavi Rathi
Enter Age: 17
Enter City: Pune
Student Added Successfully!

===== STUDENT DATABASE =====
1. Add Student
2. Search Student
3. Display All Students
4. Exit
Enter Your Choice: 3

===== ALL STUDENTS =====
-----
Roll No : 101
Name   : Vaishnavi Rathi
Age    : 17
City   : Pune

===== STUDENT DATABASE =====
1. Add Student
2. Search Student
3. Display All Students
4. Exit
Enter Your Choice: 4
Thank You!
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """
```

```
'''
```

Q5. (Dictionaries + Functions + Control Structures)

Write a function `student_database()` that uses a dictionary (roll number as key) to store student records.

Menu:

1. Add Student

2. Search Student
3. Display All Students
4. Exit

Use get(), update(), and exception handling.
'''

Function to manage student database

```
def student_database():
```

```
    # Dictionary to store student records
```

```
    students = {}
```

```
    while True:
```

```
        print("\n===== STUDENT DATABASE =====")
```

```
        print("1. Add Student")
```

```
        print("2. Search Student")
```

```
        print("3. Display All Students")
```

```
        print("4. Exit")
```

```
    try:
```

```
        choice = int(input("Enter Your Choice: "))
```

```
        # Add Student
```

```
        if choice == 1:
```

```
            roll = int(input("Enter Roll Number: "))
```

```
            name = input("Enter Name: ")
```

```
            age = int(input("Enter Age: "))
```

```
            city = input("Enter City: ")
```

```
            # Store data using update()
```

```
            students.update({
```

```
                roll: {
```

```
                    "name": name,
```

```
                    "age": age,
```

```
                    "city": city
```

```
                }
```

```
            })
```

```
            print("Student Added Successfully!")
```

```
        # Search Student
```

```
        elif choice == 2:
```

```
            roll = int(input("Enter Roll Number: "))
```

```
            student = students.get(roll)
```

```

if student:

    print("\n===== STUDENT DETAILS =====")
    print("Roll No :", roll)
    print("Name   :", student["name"])
    print("Age    :", student["age"])
    print("City   :", student["city"])

else:
    print("Student Not Found!")

# Display All Students
elif choice == 3:

    if not students:
        print("No Student Records Found.")

    else:

        print("\n===== ALL STUDENTS =====")

        for roll, student in students.items():

            print("-----")
            print("Roll No :", roll)
            print("Name   :", student["name"])
            print("Age    :", student["age"])
            print("City   :", student["city"])

# Exit
elif choice == 4:

    print("Thank You!")
    break

else:
    print("Invalid Choice!")

except ValueError:
    print("Error: Please enter valid input.")

# Call the function
student_database()

```

"Explanation in Comments"

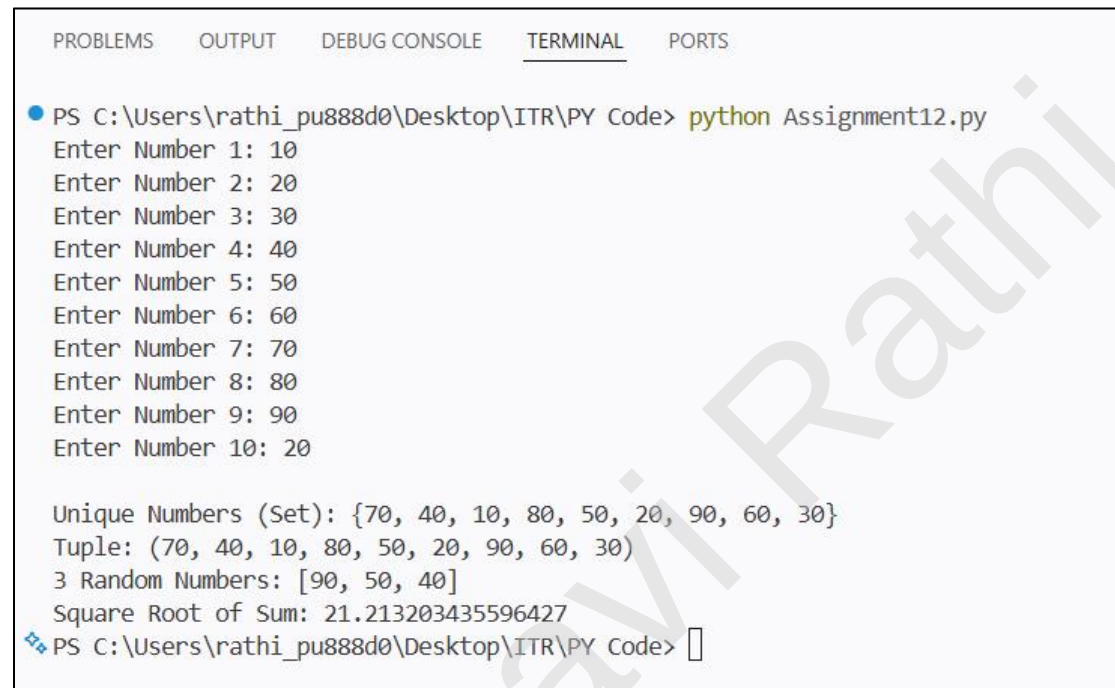
A dictionary is used to store student records.

Roll number is used as the key.
update() adds new student records.
get() searches a student safely without causing errors.
A while loop keeps displaying the menu until Exit is selected.
Exception handling prevents crashes due to invalid input.

Vaishnavi Rathni

Q6. Sets + Tuples + Modules

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py
Enter Number 1: 10
Enter Number 2: 20
Enter Number 3: 30
Enter Number 4: 40
Enter Number 5: 50
Enter Number 6: 60
Enter Number 7: 70
Enter Number 8: 80
Enter Number 9: 90
Enter Number 10: 20

Unique Numbers (Set): {70, 40, 10, 80, 50, 20, 90, 60, 30}
Tuple: (70, 40, 10, 80, 50, 20, 90, 60, 30)
3 Random Numbers: [90, 50, 40]
Square Root of Sum: 21.213203435596427
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """
```

```
'''
```

Q6. (Sets + Tuples + Modules)

Write a program that:

1. Takes 10 numbers as input and stores unique numbers in a set.
2. Converts the set into a tuple.
3. Uses random module to pick and print 3 random numbers.
4. Uses math module to print the square root of the sum of tuple elements.
5. Handle possible exceptions.

```
'''
```

```
# Import required modules
```

```
import random
```

```
import math
```

```
try:
```

```

# Create an empty set
numbers = set()

# Take 10 numbers as input
for i in range(1, 11):
    num = int(input(f"Enter Number {i}: "))
    numbers.add(num)

# Convert set to tuple
numbers_tuple = tuple(numbers)

print("\nUnique Numbers (Set):", numbers)
print("Tuple:", numbers_tuple)

# Select 3 random numbers from the tuple
if len(numbers_tuple) >= 3:
    random_numbers = random.sample(numbers_tuple, 3)
    print("3 Random Numbers:", random_numbers)
else:
    print("Less than 3 unique numbers available.")

# Calculate square root of the sum
total = sum(numbers_tuple)

if total >= 0:
    print("Square Root of Sum:", math.sqrt(total))
else:
    print("Square root cannot be calculated for a negative sum.")

except ValueError:
    print("Error: Please enter valid integer values.")

except Exception as e:
    print("Error:", e)

"""Explanation in Comments"""

# A set stores only unique numbers.
# The set is converted into a tuple.
# random.sample() selects 3 random numbers from the tuple.
# sum() calculates the total of tuple elements.
# math.sqrt() calculates the square root of the total.
# Exception handling prevents the program from crashing due to invalid input.

```

Q7. Lambda + range() + Lists + Exception Handling

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py
Squares of Numbers:
[1, 4, 9, 16, 25, 36, 49, 64, 81, 100, 121, 144, 169, 196, 225, 256, 289, 324, 361, 400]

Even Squares:
4
16
36
64
100
144
196
256
324
400
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """
```

```
'''
```

Q7. (Lambda + range()) + Lists + Exception Handling)

Create a lambda function to calculate the square of a number.

Write a program that:

1. Uses range(1, 21) to generate numbers.
2. Stores the squares of these numbers in a list using the lambda function.
3. Prints only the even squares from the list.
4. Add exception handling if needed.

```
'''
```

```
try:
```

```
# Lambda function to calculate square
square = lambda num: num ** 2
```

```
# List to store squares
squares = []
```

```
# Generate numbers from 1 to 20 and store their squares
for num in range(1, 21):
    squares.append(square(num))

# Display all squares
print("Squares of Numbers:")
print(squares)

# Display even squares
print("\nEven Squares:")
for value in squares:
    if value % 2 == 0:
        print(value)

except Exception as e:

    # Handle unexpected errors
    print("Error:", e)

"Explanation in Comments"

# The lambda function calculates the square of a number.
# range(1, 21) generates numbers from 1 to 20.
# The squares are stored in a list.
# A loop checks each square and prints only the even ones.
# Exception handling is added to catch any unexpected errors.
```

Q8.Tuples + Dictionaries + OOP

Output Screenshot:



```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py
Employee ID : 101
Name       : Vaishnavi Rathi
Department : IT
Salary     : 55000.0

===== EMPLOYEE DETAILS =====
Employee ID : 102
Name       : Svanandi Shimpi
Department : Finance
Salary     : 60000.0

===== EMPLOYEE DETAILS =====
Employee ID : 103
Name       : Lavanya Tribhuvan
Department : HR
Salary     : 85000.0
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """

"""
Q8. (Tuples + Dictionaries + OOP)

Create a class Employee with:
1. emp_id
2. name
3. details (tuple containing department and salary)

Method:
show_details()

Create a dictionary where employee ID is the key
and Employee object is the value.

Add 3 employees and display all using a loop.
"""
```

```

# Employee class
class Employee:

    # Constructor to initialize employee details
    def __init__(self, emp_id, name, department, salary):
        self.emp_id = emp_id
        self.name = name
        self.details = (department, salary)

    # Method to display employee details
    def show_details(self):

        print("\n===== EMPLOYEE DETAILS =====")
        print("Employee ID :", self.emp_id)
        print("Name      :", self.name)
        print("Department :", self.details[0])
        print("Salary    :", self.details[1])

# Dictionary to store employee objects
employees = {}

try:

    # Add 3 employees
    for i in range(1, 4):

        print(f"\nEnter Details of Employee {i}")

        emp_id = int(input("Enter Employee ID: "))
        name = input("Enter Name: ")
        department = input("Enter Department: ")
        salary = float(input("Enter Salary: "))

        # Create Employee object
        employee = Employee(emp_id, name, department, salary)

        # Store object in dictionary
        employees[emp_id] = employee

    # Display all employees
    print("\n===== ALL EMPLOYEES =====")

    for employee in employees.values():
        employee.show_details()

except ValueError:

    # Handle invalid input
    print("Error: Please enter valid input.")

```

"Explanation in Comments"

- # The Employee class stores employee information.
 - # Department and salary are stored together as a tuple.
 - # Employee objects are stored in a dictionary using employee ID as the key.
 - # A loop is used to add 3 employee records.
 - # Another loop displays all employee details.
 - # Exception handling prevents invalid user input.
-

Vaishnavi Rathni

Q9.Strings + Sets + Exception Handling + Modules

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
● PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py
Enter a sentence: Python is Easy and High-level Language...

===== UNIQUE WORDS =====
and
easy
high-level
is
language...
python

Total Unique Words : 6
Square of Total : 36.0
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """

"""
Q9. (Strings + Sets + Exception Handling + Modules)

Write a program that:
1. Takes a sentence as input.
2. Extracts all unique words using a set.
3. Prints the unique words in sorted order.
4. Uses the math module to print the square of the total
   number of unique words.
5. Handle exceptions gracefully.
"""

# Import math module
import math

try:

    # Take sentence input
```



```

sentence = input("Enter a sentence: ")

# Check for empty input
if sentence.strip() == "":
    raise ValueError("Sentence cannot be empty.")

# Convert sentence into words
words = sentence.lower().split()

# Store unique words in a set
unique_words = set(words)

# Sort unique words
sorted_words = sorted(unique_words)

# Display unique words
print("\n===== UNIQUE WORDS =====")
for word in sorted_words:
    print(word)

# Count unique words
total = len(unique_words)

# Print square using math module
print("\nTotal Unique Words :", total)
print("Square of Total   :", math.pow(total, 2))

except ValueError as e:

    # Handle empty sentence
    print("Error:", e)

except Exception as e:

    # Handle unexpected errors
    print("An error occurred:", e)

```

"""Explanation in Comments"""

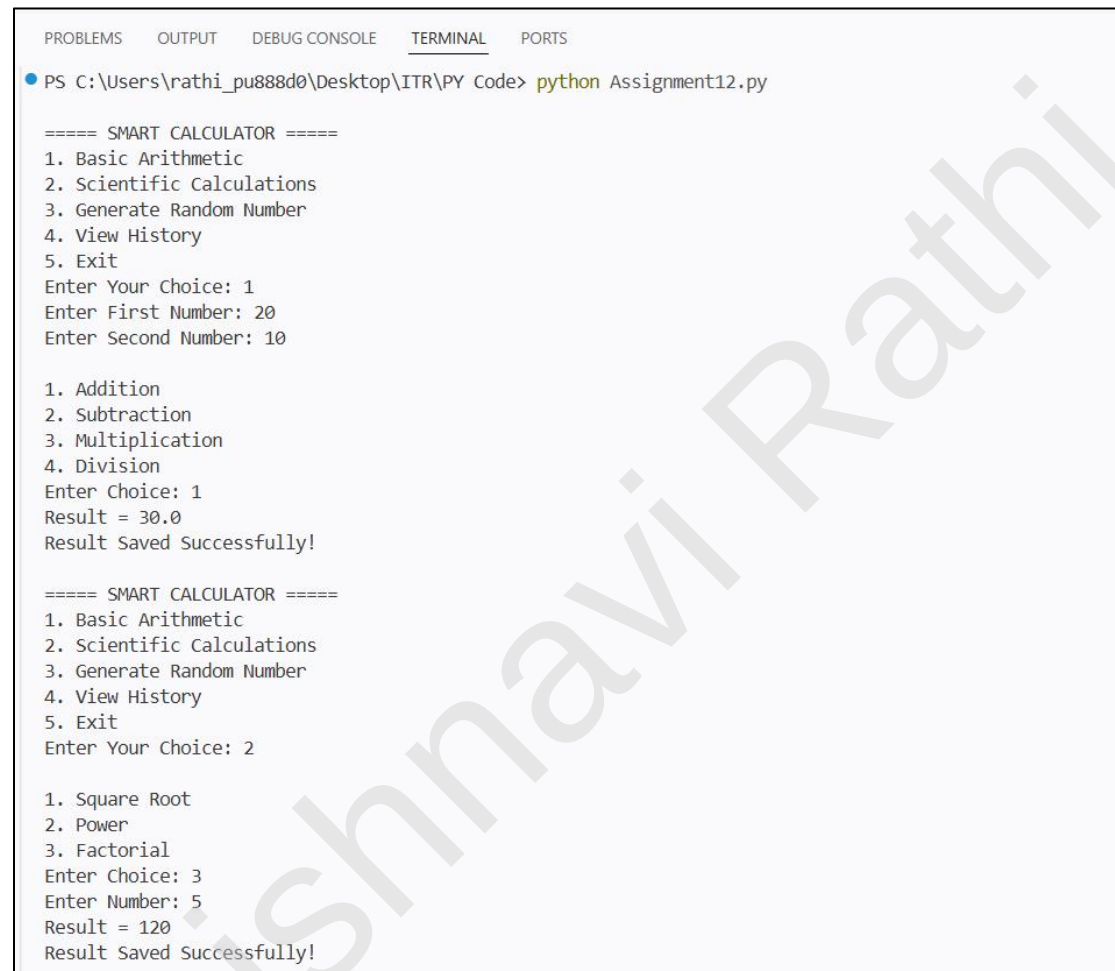
```

# The program accepts a sentence from the user.
# split() converts the sentence into a list of words.
# A set stores only unique words.
# sorted() arranges the words alphabetically.
# len() counts the total unique words.
# math.pow() calculates the square of the total unique words.
# Exception handling manages empty input and unexpected errors.

```

Q10. Mini Project - Comprehensive

Output Screenshot:



```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py

===== SMART CALCULATOR =====
1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Number
4. View History
5. Exit
Enter Your Choice: 1
Enter First Number: 20
Enter Second Number: 10

1. Addition
2. Subtraction
3. Multiplication
4. Division
Enter Choice: 1
Result = 30.0
Result Saved Successfully!

===== SMART CALCULATOR =====
1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Number
4. View History
5. Exit
Enter Your Choice: 2

1. Square Root
2. Power
3. Factorial
Enter Choice: 3
Enter Number: 5
Result = 120
Result Saved Successfully!
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> python Assignment12.py
===== SMART CALCULATOR =====
1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Number
4. View History
5. Exit
Enter Your Choice: 3
Enter Starting Number: 1
Enter Ending Number: 100
Random Number = 30
Result Saved Successfully!

===== SMART CALCULATOR =====
1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Number
4. View History
5. Exit
Enter Your Choice: 4

===== CALCULATION HISTORY =====
Time : 26-06-2026 23:00:18
Result : 30.0
-----
Time : 26-06-2026 23:00:35
Result : 120
-----
Time : 26-06-2026 23:00:56
Result : 30
-----

===== SMART CALCULATOR =====
1. Basic Arithmetic
2. Scientific Calculations
3. Generate Random Number
4. View History
5. Exit
Enter Your Choice: 5
Thank You!
PS C:\Users\rathi_pu888d0\Desktop\ITR\PY Code> 
```

Program Code:

```
""" ASSIGNMENT 12 """
```

```
'''
```

Q10. (Mini Project - Comprehensive)

Create a Smart Calculator & Data Manager using:

1. Functions
2. Exception Handling
3. math module
4. random module
5. Dictionary
6. While Loop Menu

```
'''
```

```

# Import required modules
import math
import random
from datetime import datetime

# Dictionary to store calculation history
history = {}

# -----
# Basic Arithmetic
# -----
def basic_arithmetic():

    try:

        num1 = float(input("Enter First Number: "))
        num2 = float(input("Enter Second Number: "))

        print("\n1. Addition")
        print("2. Subtraction")
        print("3. Multiplication")
        print("4. Division")

        choice = int(input("Enter Choice: "))

        if choice == 1:
            result = num1 + num2

        elif choice == 2:
            result = num1 - num2

        elif choice == 3:
            result = num1 * num2

        elif choice == 4:
            result = num1 / num2

        else:
            print("Invalid Choice!")
            return

        print("Result =", result)

        save_result(result)

    except ValueError:
        print("Error: Invalid Input.")

```

```

except ZeroDivisionError:
    print("Error: Division by Zero is not allowed.")

# -----
# Scientific Calculator
# -----
def scientific_calculation():

    try:

        print("\n1. Square Root")
        print("2. Power")
        print("3. Factorial")

        choice = int(input("Enter Choice: "))

        if choice == 1:

            num = float(input("Enter Number: "))
            result = math.sqrt(num)

        elif choice == 2:

            base = float(input("Enter Base: "))
            power = float(input("Enter Power: "))
            result = math.pow(base, power)

        elif choice == 3:

            num = int(input("Enter Number: "))
            result = math.factorial(num)

        else:
            print("Invalid Choice!")
            return

        print("Result =", result)

        save_result(result)

    except ValueError:
        print("Error: Invalid Input.")

    except Exception as e:
        print("Error:", e)

# -----
# Generate Random Numbers

```

```

# -----
def generate_random():

    try:

        start = int(input("Enter Starting Number: "))
        end = int(input("Enter Ending Number: "))

        number = random.randint(start, end)

        print("Random Number =", number)

        save_result(number)

    except ValueError:
        print("Error: Invalid Input.")

# -----
# Save Result
# -----
def save_result(result):

    timestamp = datetime.now().strftime("%d-%m-%Y %H:%M:%S")

    history.update({
        timestamp: result
    })

    print("Result Saved Successfully!")

# -----
# View History
# -----
def view_history():

    if not history:

        print("No History Available.")
        return

    print("\n===== CALCULATION HISTORY =====")

    for time, result in history.items():

        print("Time :", time)
        print("Result :", result)
        print("-" * 30)

```

```

# -----
# Main Menu
# -----
while True:

    print("\n===== SMART CALCULATOR =====")
    print("1. Basic Arithmetic")
    print("2. Scientific Calculations")
    print("3. Generate Random Number")
    print("4. View History")
    print("5. Exit")

    try:

        choice = int(input("Enter Your Choice: "))

        if choice == 1:
            basic_arithmetic()

        elif choice == 2:
            scientific_calculation()

        elif choice == 3:
            generate_random()

        elif choice == 4:
            view_history()

        elif choice == 5:
            print("Thank You!")
            break

        else:
            print("Invalid Choice!")

    except ValueError:
        print("Error: Please Enter Numbers Only.")

```

"""Explanation in Comments"""

```

# Functions are used for different operations.
# Basic arithmetic performs +, -, *, /.
# Scientific calculations use the math module.
# Random numbers are generated using the random module.
# Dictionary stores results with timestamp as key.
# Exception handling prevents program crashes.
# While loop keeps displaying the menu until Exit.

```
